基于重连续化的反射镜优化调控

摘要

本文基于“FAST”中国天眼射电望远镜的客观物理结构以及物理调控机制：通过牵引索调控主索点进行位置变化，从而调控位于主索上的反射镜面进行倾角变化，从而实现信号的反射调节。通过解决本问题，可以将本调控优化算法应用于各种离散位点向目标形状调控的问题中。

针对问题一，主要解决抛物线顶点的径向伸缩量最优化问题。根据抛物线光学性质，我们只要求解出抛物线顶点到馈源舱的最优化距离也就就解决了抛物面的方程问题

针对问题二，是一个最优化调控问题。我们需要寻找到各个调控节点径向伸缩多少距离才可与理想抛物线“最贴近”，**我们采取纳入第一问的顶点伸缩参量一起实现调控最优化**。

先进行三维线性**空间旋转**变化，将坐标系旋转到以基准球心为原点，以抛物面主轴为 z 轴的坐标系中，又由于主索点三角短程连接必然导致各个节点呈现“圈层分布”，所以我们对同一 z 值的节点**降维平面化**为二维平面直角坐标系上第一象限上的一系列离散点。

由于我们提取出来的点数目过多，对每一个点进行优化调控会出现算量过大，所以我们采用**重离散化配合插值拟合**的方法欲得到调控伸缩量的一个**最优调控函数拟合式**，这样我们带入对应自变参量就可以得到我们对应点的伸缩调控值。

我们先定向用**非线性最小二乘法**拟合第一象限二维离散点基准圆方程，后只取第一象限中对圆心偏角为30°的劣弧按照对圆心偏角等分100份**重离散化**取点（最终有效口径为300m，对圆心60°，由于调控的对称性我们只研究一半就可以）

以这 100 个点的径向伸缩量和顶点的伸缩量为 101 个优化参数运用**遗传算法**进行优化，并对优化求解出来的Δr与极坐标系中各个点对应的偏角 θ 和各个点的新坐标 y 与 x 之间进行**插值拟合**，得到拟合函数式，再把空间点对应角参数带入，得到各个点的伸缩调控量。并将新坐标逆空间变换回原坐标系中，得到结果。

第二问结果储存于**支撑文件的附录四**中，并且优化得出抛物线顶点径向背离圆心 0.38994m

第一问理想抛物线方程为：

$$x^2+y^2=4(0.466R+0.38994)z+300.78994，R=300.4$$

针对问题三，主要解决抛物面接受比的评估。我们运用几何知识，继续在第二问构建的降维平面上处理，并求解反射镜面上每一个入射点x_0对应的反射光线到达馈源舱同轴向高度时，反射线上对应点的横坐标x_1值，若x_1位于馈源舱周围 0.5m 范围内，这个x_0点为有效点，我们以步长 1mm 划分 x∈（0,150m）为 1500000 份，由于间隙过密，故这个离散可以高度估计连续的情况，最后用有效点数目比上所有点数目就是我们的接收比。

馈源舱工作抛物面接受比为：12.2094%

馈源舱基准反射球面接收比为：5.45084%

本模型算法创新点在于：1，并未直接计算大量真实节点的调控参数，而是转为求解最优调控情况下的调控函数拟合式，最后再反带入求解调控参数，这节省了大量算力并，100 个离散点也具有较好精度 2，我们同时考虑了抛物线形状（顶点牵引）与其他调控点的共同调控

关键词：空间旋转变换，降维平面化，非线性最小二乘法，重离散化，遗传优化算法，插值拟合

一， 问题重述

1.1 问题背景

“FAST”天眼望远境主要由主动反射面系统，信号接受系统（馈源舱）以及各种支撑连接系统组成，我们用地锚点上的促动器通过牵引索对三角短程连接的主索网上主索点位置进行调控，而反射镜就放置于三角短程连接的主索点构成的三角区域上。

从而，我们通过调节牵引索长度变化，进一步调控主索点位置，从而使各个反射镜面的水平夹角发生变化，从而构成不同形态的反射曲面，实现我们需要的信号聚焦。

1.2 问题提出

（1）问题一欲求解出理想抛物面方程，我们设置抛物线顶点牵引移动量为参数进行优化

（2）问题二欲求解调控后的各个调控节点的伸缩量，以及各个主索点伸缩完之后的位置坐标

（3）问题三欲分析建立起来的调控反射模型的信号接收比

二， 问题分析

2.1 问题一，二

问题一中给出天体方位α=0°，β=90°的条件，要求结合反射板调节因素确定理想抛物面。基于β=π/2，α=0°，且观测目标S、圆心C和焦点P固定，故可确定出的理想抛物面方程，由此我们引入RMSE公式，以确定优化目标并以Δr为最优化控制变量。接着考虑到主索点变化的对称性与一致性，我们再通过空间降平面化处理，通过对离散点的拟合与重离散化。将重离散化后的n个点通过遗传算法进行优化，求解出该n个优化参数后便可确定理想抛物面的Δr，并将其代回抛物面方程便可得到我们的理想抛物面方程。

问题二中给出天体方位α=36.765°，β=78.169°的条件，要求确定理想抛物面并给出各促动器的伸缩量，使得调节相关的促动器伸缩量能使反射面尽量贴近该理想抛物面。考虑到倾斜状态下的问题处理较为复杂，我们在第一问的基础上先将坐标空间进行旋转变换以简化问题的求解，再重复问题一中的上述各操作，便可确定理想抛物面方程并通过已有的n个离散点的x,y,z值进行插值拟合$r(z)$，$y(x)$确定并输出各主索节点位置坐标以及各促动器的伸缩量。

2.2 问题三

问题三中要求在基于第二问反射面调节方案的前提下，计算馈源舱有效区域接收到的反射信号与300米口径内反射面的反射信号之比，并与基准反射球面的接收比作比较。针对问题三，考虑到主索点变化的对称性与一致性以及简化复杂的空间问题，我们首先采用与一、二问同样的空间降平面化处理，并旋转坐标空间使入射电磁波等效为垂直入射，进而为确定符合接收条件的反射点y值，我们可建立等效射线模型。由于对称性我们可用0-150米的接受比替代300米口径的接收比，进而再对0-150m间的点进行离散处理，在众多离散点中找出符合终端y值的点，以符合条件的点与全部点之比作为接受比并求解。

三， 模型假设

1，牵引索仅径向牵引，使主索点仅在极坐标系中对应径向伸缩
2，电磁波传播过程不受干扰，以纯平行于馈源舱和球心连线方向射入反射镜面

四， 符号说明

符号	说明	
α	天体 S 相对于基准球面球心的方位角	°
β	天体 S 相对于基准球面球心的仰角	°
Δr	顶点伸缩量	m
F	焦面与基准球面的半径差 0.466R	m
$\Delta r(\theta)$	Δr 伸缩量关于转角 θ 的拟合函数	°
$y(x)$	索引点的 y 值关于 x 的拟合函数	m
R_z	绕 z 轴旋转的正交矩阵	1
R_y	绕 y 轴旋转的正交矩阵	1
$A_{原基}$	原坐标系的向量基底	1
d	理想抛物线的纵轴截距	m
x'	降平面化后的 x 坐标，即原 $\sqrt{x^2+y^2}$	m
y'	降平面化后的 y 坐标，即原 z	m
h_0	馈源舱与球心的高度差	m
y_0	反射点的纵坐标	m

注：其余符号已经在文章中注解

五， 问题一的求解

5.1 电磁信号全聚焦的条件

由于题目已经告知我们工作有效反射面为抛物线型，又我们的馈源舱为信号接收点，为了尽可能多的电磁信号可以反射到馈源舱，我们考虑抛物线的光学性质：

设抛物线的方程为：$y2=2px$，$p>0$。

从点F出发的入射光线与抛物线的交于点$M\ (x0,y0)$。

MT为在点M处的切线，MN为点M的法线。反射光线为MQ。

则根据物理学的光线反射定律，有$\angle FMN=\angle QMN$。

其中切线MT起到了反射镜的作用。

对抛物线的方程求导，有

$$2y\cdot y'=2p$$

$$y'=\frac{p}{y}$$

其中利用了隐函数的求导法则。则在点M处的导数值为

$$y'_M=\frac{p}{y_0}$$

于是过点M的法线MN的斜率为

$$k_{MN}=-\frac{1}{k_{MT}}=-\frac{y_0}{p}$$

法线MN的方程为:

$$y-y_0=-\frac{y_M}{p}(x-x_0)$$

令上式的$y=0$，得到点N的横坐标:

$$x_N=x_0+p$$

又焦点F为$(p/2,0)$，则NF的长度为:

$$NF=x_0+\frac{p}{2}$$

根据抛物线的定义，点M到点F的距离等于它到准线l的距离。

准线的方程为$l:x=-p/2$。于是有，

因此有:

$$NF=MF$$

在等腰三角形FMN中

∠FMN=∠FNM

而根据光线反射定律，有

∠FMN=∠QMN

所以

∠FNM=∠QMN

根据内错角相等，可得$MQ\parallel FN$，即反射光线平行于x轴。

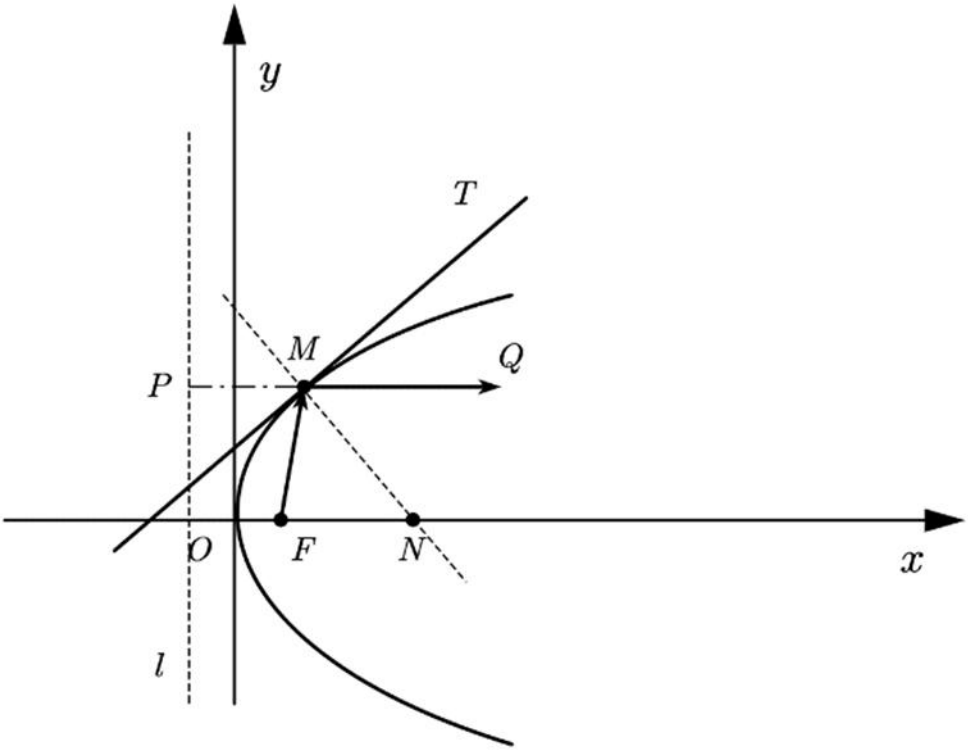


图 1，抛物线光学性质反射

由于反射的光路可逆性，我们可知平行于主光轴的入射平行光线将汇聚于焦点上。

故我们要通过调控，使馈源舱尽可能位于我们的抛物线焦点上。

5.2 顶点的调控参数控制

只要我们确定了顶点处的伸缩量，我们就可以确定理想抛物线的焦半径：

$$\frac{p}{2} = 0.466R + \Delta r$$

$$x^2 + y^2 = 4(0.466R + \Delta r)z + d$$

我们采用最优化算法中的遗传算法，纳入定点的伸缩量为优化参数进行优化。

由于具体过程与问题二除去空间旋转变换过程外一致，故我们此处仅仅陈述建模以及过程和结果，具体详解请详见问题二：

（1） 降维平面化为二维第一象限离散点（但符合基准圆面规律）

（2） 非线性最小二乘法拟合 R 具体值得到方程

（3） 重离散化并且设置 101 个优化参数，建立优化向量

（4） 遗传算法优化 101 个参数

通过上述过程我们得到的优化解为：

顶点伸缩量为：△r=0.38994

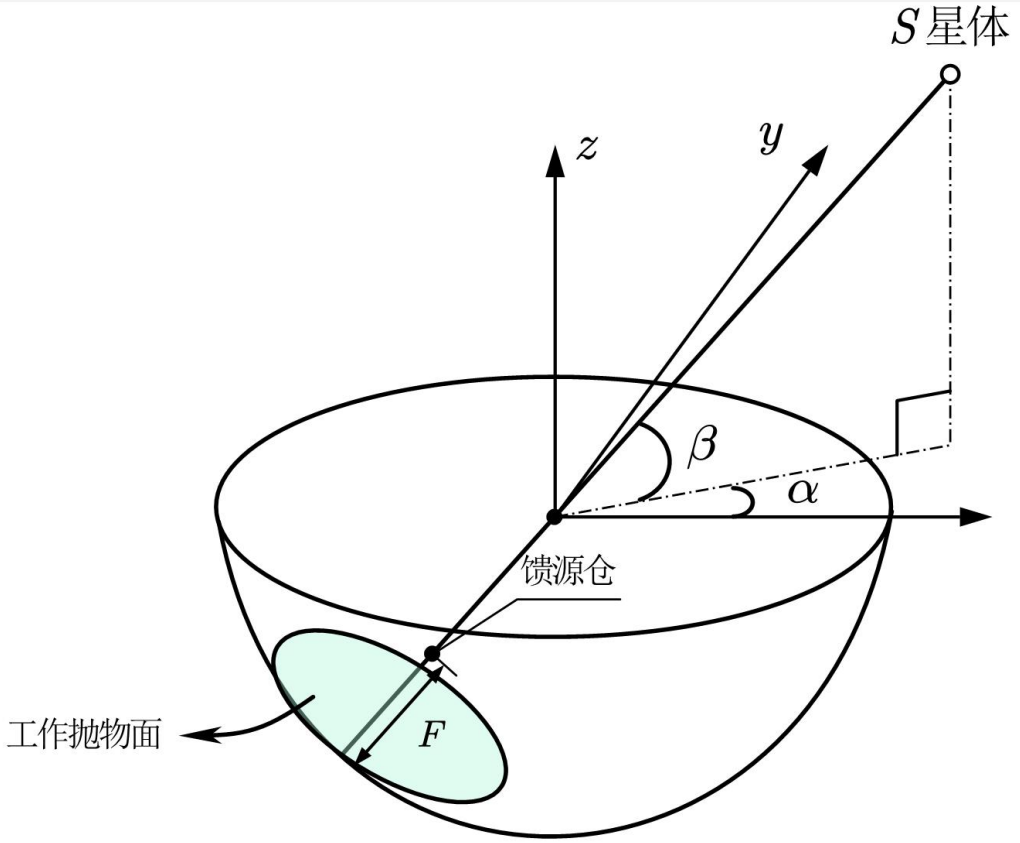
理想抛物线方程为：

$$x^2 + y^2 = 4(0.466R + 0.38994)z + 300.78994$$

六， 问题二的求解

6.1 空间旋转变换

由于第二问给予的观测星体方位偏离我们的竖直垂直主轴，而为了使电磁信号可以平行射入我们的有效工作区域，馈源舱、基准球心、观测星体三者应当处于共线位置。而为了实现反射信号的全聚焦，我们的工作抛物面主轴也同样应当与上述直线共轴，并且要尽量保证馈源舱位于工作抛物面的焦点位置，这样就可以实现较好的电磁聚焦效果。



但是我们最终是要调节空间中的某些主索节点的位置来实现基准球面到工作抛物面的转化，而处于默认坐标系下，工作抛物面最终是倾斜的，虽然我们的牵引索都是径向伸缩，但是倾斜情况下：第一：方程形式不好处理，第二：调节位点不好控制以及确定，第三：倾斜坐标系下一切运算以及计算都要叠加上多于的难度与操作

所以我们采取先对坐标空间进行旋转空间变换，但是保证了以下几点机制在旋转变换前后不变：

1，以球心为坐标系原点，保证了各个牵引索在空间变换之后伸缩机制仍是径向伸缩

2，旋转前后各个点间的距离不变，即旋转前后欧式距离不变，空间变换为正交变换

那么我们现在就以如下指标为目标进行坐标系空间变换：以球心为坐标系原点，以抛物面主轴为坐标系 z 轴，S 星体与球心连线位于 XoZ 平面上（由此来确立 x 轴与 y 轴位置）我们运用正交矩阵来对我们的数据进行正交变换，转换坐标系为我们想要的坐标系：

我们先逆时针旋转z轴$\alpha = 36.795°$

$$R_z = \begin{pmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

经过次变换，我们将S轴的牵垂投影x'轴转换到与x轴重合位置

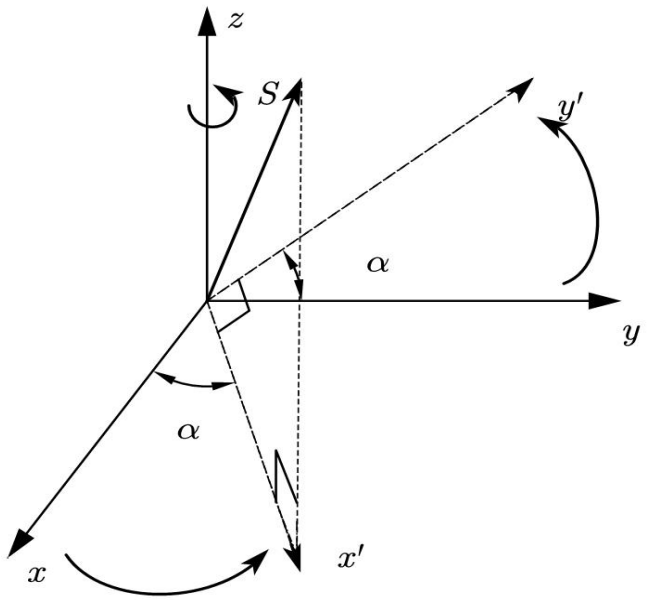


图 1，转动 z 轴消除 α 影响

再绕（从y轴正向看）y轴逆时针转动（$90° - \beta$)

使z轴与S轴重合

$$R_y = \begin{pmatrix} \cos(90° - \beta) & 0 & \sin(90° - \beta) \\ 0 & 1 & 0 \\ -\sin(90° - \beta) & 0 & \cos(90° - \beta) \end{pmatrix}$$

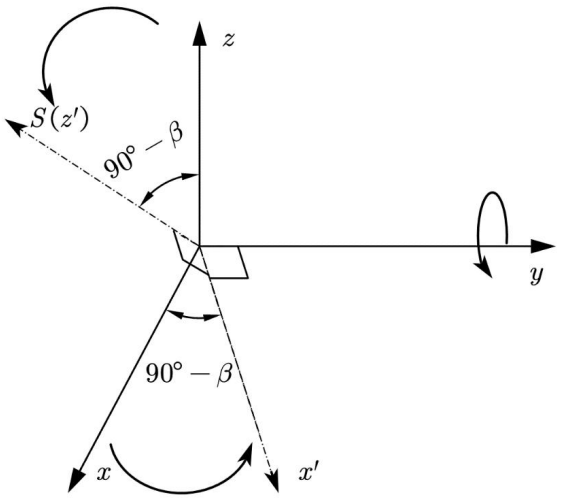


图 2，转动 y 轴消除 β 影响

故而有坐标系转换后坐标间的如下转换关系：

$$R_{\mathrm{y}} \times R_z \times A_{\text{原基}} = A_{\text{新基}}$$

$$A_{\text{新基}} \times X_{\text{新坐标}} = A_{\text{原基}} \times X_{\text{原坐标}}$$

$$R_{\mathrm{y}} \times R_z \times A_{\text{原基}} \times X_{\text{新坐标}} = A_{\text{原基}} \times X_{\text{原坐标}}$$

等式两边同乘新基的逆矩阵

$$X_{\text{新坐标}} = (R_{\mathrm{y}} \times R_z \times A_{\text{原基}})^{-1} \times A_{\text{原基}} \times X_{\text{原坐标}}$$

$$X_{\text{新坐标}} = A_{\text{原基}}{}^{-1} \times (R_{\mathrm{y}} \times R_z)^{-1} \times A_{\text{原基}} \times X_{\text{原坐标}}$$

由于$A_{\text{原基}}$为标准基故结果变化为：

$$X_{\text{新坐标}} = R_z{}^{-1} \times R_{\mathrm{y}}{}^{-1} \times X_{\text{原坐标}}$$

通过上述转换关系，我们成功的转换坐标系为以抛物线主轴为 z 轴，以球心为原点的坐标轴。

6.2 空间降维平面化处理

由上述空间变换，我们已经实现了将馈源舱以及工作抛物面“摆正”的步骤，但由于旋转抛物面的对称性，以及径向伸缩调控的对称性，我们会发现：同 z 值（即同高度圆周截面）上的主索点牵引索变化是相同的，那么我们若同时以三维坐标系纳入考虑这些相同值的点，难免会出现算力浪费的情况。

为了更高效方便快速的计算，我们选择“合并同类项”的处理方法，由于主索点的排布规律：

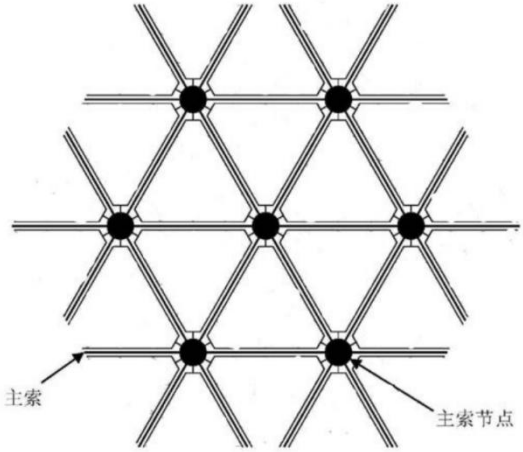


图 3，主索点连接示意图

主索点的“三角连接”构建出固定的“晶胞连接”，以及最后收敛连接于同一个最低点的性质导致其所有主索点必然“分层”：

以最低中心点/圈为第 0 圈，

与第 0 圈直接相连却不属于第 0 圈的点为第 1 圈，

与第 1 圈直接相连却不属于第 1 圈与第 0 圈的点归属于第 2 圈，皆以此类推。

而由于最终工作抛物面或者基准圆面要保证旋转对称性也就是水平各向同性

而各个曲面又是由离散的主索点作为节点构建出来的，故而各个主索点也同样符合水平各向同性，所以各个“同圈节点”必然对应着同高度同 z 值。
这从而给我们降维平面化提供了理论支撑。

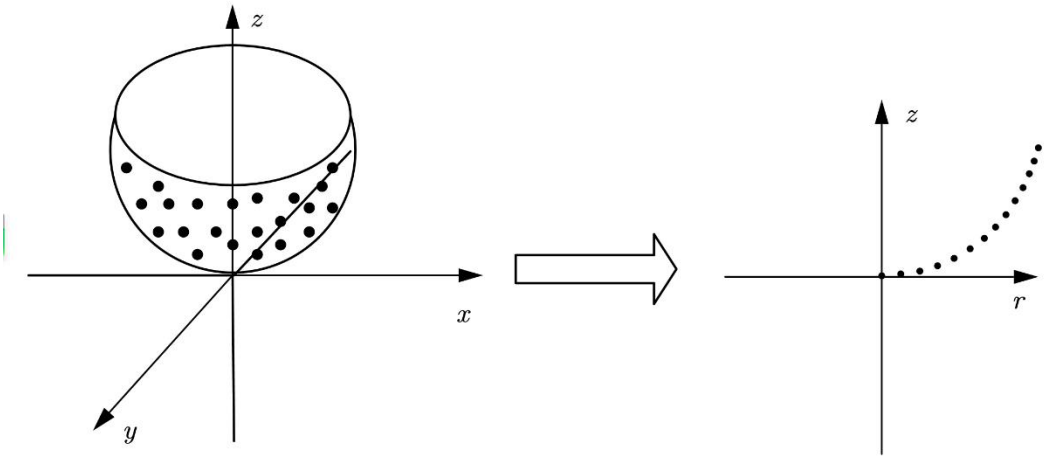


图 4，降维平面化示意图

那么我们现在就来进行降维平面化也就是“合并同类项”，

首先运用MATLAB的索引

运用索引将所有同z值的点都只当做一个点处理

并且进行如下代替

y′=z

$x' = r = \sqrt{x^2 + y^2}$

由于转换后的x'表示各个节点到中心主轴的距离，并且明显x'大于 0，y 即为 z 坐标，故而我们通过平面降维化将三维问题降维为了一个更容易处理的二维问题，并且此二维化纯归属于第一象限：

6.3 重离散化得优化向量

6.3.1 非线性最小二乘拟合

我们根据上面的分析，已经将离散的点归属降维为了二维平面第一象限上的点但是我们已知其为基准圆方程，并且其为基准圆方程第一象限下半截：故而有了其方程表达式：

$$y = R - \sqrt{R^2 - x^2}$$

我们现在即利用目标含参方程式，平面降维化离散数据以及非线性最小二乘法进行拟合：

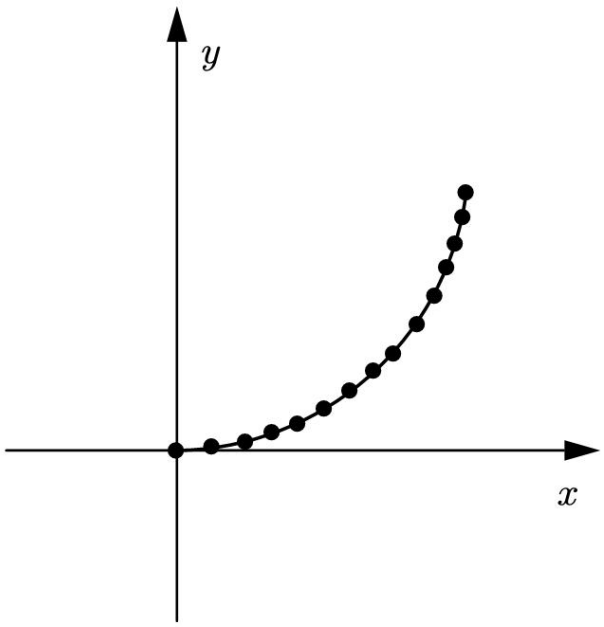


图 5，拟合示意图

6.3.1.1 定义误差函数

我们欲求解函数表达式为：

$$y = R - \sqrt{R^2 - x^2}$$

非线性最小二乘法的内涵即为寻找到对应的参数 R，使我们已有的数据点与对应函数表达式之间的距离达到最小，我们通过引入误差函数来实现对“距离”的度量：

$$S = \sum_{i=1}^{m} (y_i - \left(R - \sqrt{R^2 - {x_i}^2}\right))^2$$

其中：$y = R - \sqrt{R^2 - {x_i}^2}$为对应点处预测值

对每个点处的真实值与预测值作差后平方和来度量“距离”

6.3.1.2 求解参数

引入了误差函数后，我们就需要求解方程式中的参数使得我们的目标误差函数达到最小值，我们可以采取优化算法，但是这里我们采用高斯-牛顿法（迭代法）将非线性方程和数据之间的关系表示为一般形式

$$f(x_i; a_1, a_2, \ldots a_m)$$

在参数值处，将上面的非线性模型围绕参数值以泰勒级数展开，并省略一阶导数后面的项。于是，对于有 1 个参数的情况有:

$$f(x_i)_{j+1} = f(x_i)_j + \frac{\delta f(x_i)_j}{\delta R} \Delta R$$

其中，下标为 j 的是初始参数值（上一次迭代的参数值），下标为 j+1 的是预测值. 于是，就将原非线性模型关于参数进行了线性化:

$$y_i - f(x_i)_j = \frac{\delta f(x_i)_j}{\delta R}\Delta R$$

以矩阵形式表示为：

$[D]=[Z_i][\Delta R]$

其中,$[Z_i]$为函数的偏导数在第j步的初始参数的值组成的矩阵

$$[Z_i]=\begin{bmatrix} \frac{\delta f1}{\delta R} \\ \frac{\delta f2}{\delta R} \\ \frac{\delta f3}{\delta R} \\ \vdots \\ \frac{\delta fn}{\delta R} \end{bmatrix}$$

其中，n为数据点的数目,$\frac{\delta fi}{\delta R}$为第i个数据点关于参数$R$的偏导

$[D]$由真实值与预测值之差组成：

$$[D]=\begin{bmatrix} y1-f(x1) \\ \vdots \\ yn-f(xn) \end{bmatrix}$$

$[\Delta R]$即为每一步的迭代步长矩阵

$[\Delta R]=[\Delta R1]$

$$\left[[Z_j]^T[Z_j]\right]\{\triangle A\}=\left\{[Z_j]^T\{D\}\right\}$$

通过该方程组我们可以求解出步长△R 从而可以得到我们参数 R 的改进值，通过重复迭代，最终达到收敛条件：

$$|\varepsilon_a|_k=\left|\frac{a_{k,j+1}-a_{k,j}}{a_{k,j+1}}\right|\times 100\%$$

计算得出 R=300.4m

故函数表达式为

$$y = 300.4 - \sqrt{300.4^2 - x^2}$$

6.3.2．1 重离散化

我们已经获取了非线性最小二乘法的拟合结果，这个是连续化的一个基准圆曲线，

但是我们要求解的是各个离散节点处的△r 调控值，但是我们又为什么要把离散结果连续化呢？

因为我们无法对 2000 多个节点同时进行参数优化，这要求太大的算力而且效率不高，我们的目的是转而求解出△r 伸缩量关于转角 θ 的函数，即我们求解的是达到工作抛物面时各个高度牵引索伸缩量的变化规律，而非某些具体点的伸缩变化量，之后只要把每个点的对应 θ 带入最终函数△r（θ）我们就有了各个节点的伸缩量

而欲求解出对应的△r（θ），我们就是采用重离散的方法，在连续的函数上再离散出 100 个点，设置这 100 个点的伸缩量为优化参数，运用后续的遗传优化算法，得到这 100 个伸缩量的最优设置，并运用这 100 个值与原始 100 个点的 θ 进行插值拟合，便得到了我们待求的△r（θ）。

我们现在进行重离散步骤：

由于抛物线的对称性，我们欲求解出△r（θ）其实只需要考虑抛物线的一半，而我们最终的有效工作区域口径只有 300m，对圆心角只有 60° 故而我们取（0,30°）部分等分 100 份，再根据角度和拟合半径 R=300.4m 计算出我们点的 x，y 坐标值，实现了重离散化。

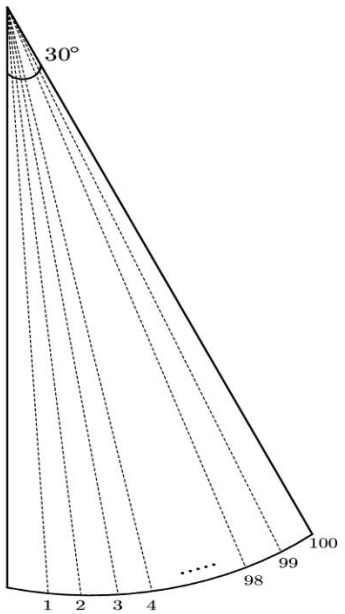


图 6，重离散化结果示意图

6.3.2.2 优化向量的设置

我们需要设置优化参数组成优化向量才能进一步进行后续的遗传算法优化。我们取 100 个重离散点的径向伸缩量为前 100 个优化参数，取抛物线顶点的伸缩平移量为第 101 个优化参数

在平面直角坐标系中理想抛物线的方程为：

$$x^2 = 4(0.466R + \Delta r)y + d$$

6.4 遗传算法优化

进行目标优化向量取值后，在伸缩长度条件限制范围内应当存在一组最优解指导

选定的样本点经过伸缩，可以使样本点所限定的目标曲线最贴近于共焦点的理想抛物线族的一支。我们采用遗传算法来进行相应优化

6.4.1 初始化

我们的最终目的是得到优化向量最终的最优化解，那么我们必然要设置优化向量中的各个参数为优化参数，由于参数数目为 101 个，这时如果用二进制编码，将面临问题：

1，目标精度每增加三位小数，二进制编码就要增加十位，精度不符合要求
2，考虑到精度要求，同时再叠加上我们已经减小算力的重离散化算法，仍就面临着解空间过大的问题

综合考虑以上两个问题：我们发现运用二进制算法会在巨大的解空间中达到一个不那么精确的解。

于是我们考虑运用浮点数编码来实现更快捷更精确的解

我们赋予 101 个优化参量每个参量为（-0.6,0.6）范围内的随机浮点数，从而符合了题目中牵引索的伸缩限制。

那么每一组优化向量就是我们需求的一个可行解，也就是组成了遗传算法的一条染色体。

6.4.2 适应度的定义

我们现在应当选定并量化优化目标数值作为遗传算法的适应度，从而通过适应度来判断我们的每组的可行解的优劣程度。

我们这里采用优化算法的目标是让最终调控样本点尽量贴近目标理想抛物线，那么我们就想到用“距离”来衡量我们的最终评价标准，也就是构成我们的优化函数目标，由于我们的各个节点是离散化的，我们很容易联想到运用离散点误差偏移距离 RMSE 作为我们的评价指标。

而我们由由优化向量的数值可以得出目标抛物线与采样点伸缩变换后坐标，从而使用 RMSE 来判断拟合效果，并作为最终输出的目标函数值，组成我们遗传算法中的适应度。

理想抛物线方程为：

$x^2 = 4(0.466R + \Delta r_{101})\ y+d$

故优化目标为：

$$\min\{RMSE\} = \min\left\{\sqrt{\frac{1}{100}\sum_{i=1}^{100}(y_{理想} - y_i)^2}\right\} = \min\left\{\sqrt{\frac{1}{100}\sum_{i=1}^{100}\left(\frac{x_i^2 - d}{4(0.466R + \Delta r_{101})} y_i\right)^2}\right\}$$

其中，(x_i, y_i)为伸缩调控后的节点坐标

d为理想抛物线纵轴截距，

Δr_{101}为抛物线顶点的径向伸缩量，也就是第101位优化参数

节点坐标计算公式为：

$$x_i = x_{i原} + \Delta r_i \sin\theta_i$$

$$y_i = y_{i原} + \Delta r_i \cos\theta$$

限制条件为：

$\Delta r_i \in (\Delta r_1, \sim\sim \Delta r_{101})$ 且 $\Delta r_i \in (-0.6, 0.6)$

由于我们的遗传优化算法设置的适应度应当是越大越好，但是我们这里应当求解的是一个最小优化，故我们应当对目标函数进行一些改写：

对于最小优化问　我们定义适应度如下：

$$f(x) = \begin{matrix} c_{max} - g(x), g(x) < c_{max} \\ 0, 其他 \end{matrix}$$

$g(x)$为我们定义的$RMSE$函数

其中c_{max}可以是一个输入值或是理论上g(x)的最大值，

或是当前所有代或最接近K代中g(x)的最大值，此时随着代数会有变化

6.4.3 交叉以及变异

将各染色体在伸缩限定数值内进行初始化，并计算初始最优染色体，接着我们便开始对染色体组进行指定步数的迭代遗传，大致分为如下几步：

(1) 变异：变异过程是遗传算法中对于解空间中执行全局搜索的步骤。对于所有染色体上的所有点位进循环遍历，并设定概率，使得这些点位有相同的概率进行相应变异，变异方式为乘以一个属于(-2,2)区间内的随机变换因子，使染色体可以在解空间中进行全局搜索。以免出现某点位超出伸缩限定范围的情况，则设计程序直接将该点位的伸缩值判定为超出的最小值或最大值。且为了满足收敛速度的需要，应当将变异概率和程度设置为随着迭代步数的增大而逐渐减小。

(2) 交叉：为了提高整体的适应度水平，增大检索出全局最优解的概率，需要染色体之间进行节点的交换，于是我们进行染色体节点的交叉。对所有染色体进行遍历并进行随机选择，将选中染色体进行随机节点的选择，将选中染色体与该染色体组中随机一条染色体的相应点位的节点进行互换。

(3) 自然选择：变异和交叉仅仅只是对于解的检索，自然选择保证了群体总体向着最优化解的方向进行收敛，并为群体中个体通过检索成为全局最优提供了更大的可能性。将染色体组按一定比例设置适应度阈值，将低于阈值的个体替换成目前染色体组的全局最优解，大幅提高了该染色体组的平均适应水平。

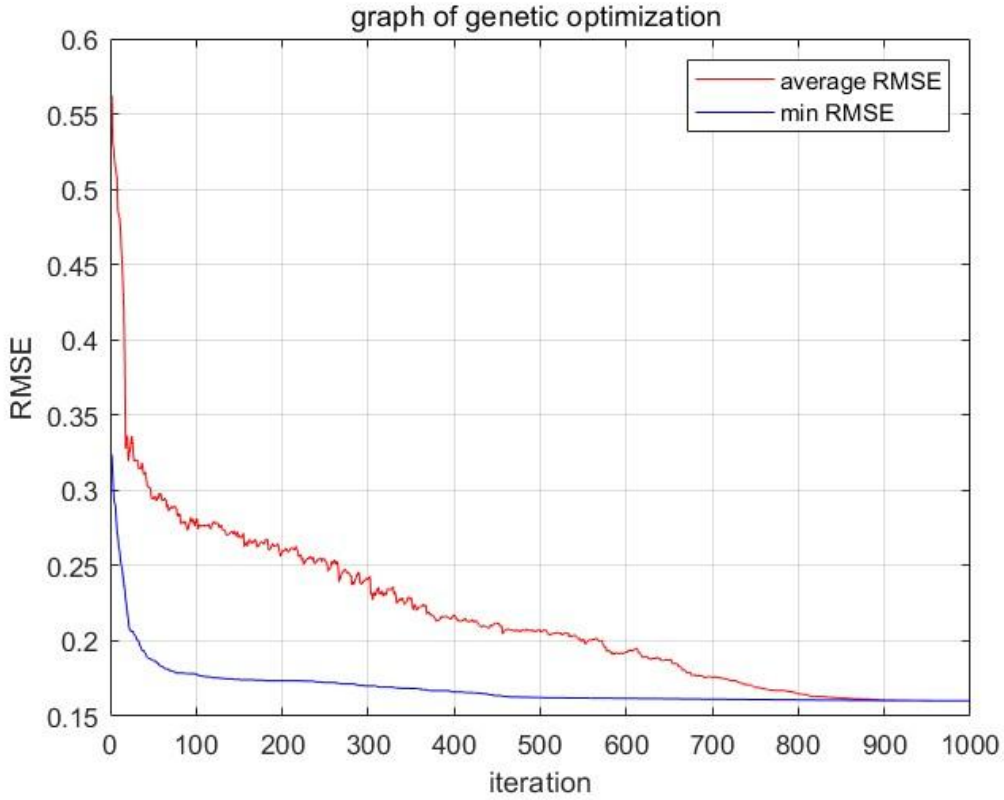


图 7，遗传优化效果图

设置迭代步数，将以上过程反复迭代便可以在最终染色体组中求选出最优染色体，也就是我们的目标最优解向量。

最终得到的最优解向量为：

最优染色体为

0.38994 :顶点向下移动距离

-0.39006	-0.38902	0.00027605	-0.38664	-0.38438
-0.38347	-0.38076	-0.37863	-0.37371	-0.36729
-0.36279	-0.3561	-0.35315	0.0028901	-0.33595
-0.32836	0.0018332	0.00033825	-0.301	7.5828e-06
-0.28214	8.0906e-08	-0.25944	-3.4632e-08	-0.23845
-0.0048554	-0.21249	0.00060346	0.0012011	0.00022
-0.15477	-0.0004365	5.6247e-05	-4.0104e-05	-0.096859
-0.0020679	-0.0022877	0.0019565	4.4833e-05	0.00070121

0.005141 -0.00099964 0.00018946 1.2188e-05 0.068222

-0.0040232 -0.00019003 0.0022793 0.10391 -0.00067619

-6.1121e-05 0.0015563 0.0081264 0.20808

0.00076694 0.24085 2.0133e-06 -6.0362e-05 0.28028

0.29528 -0.00015435 -1.3051e-05 0.32908 0.33749

0.3458 0.35275 0.35794 0.36446 0.36803

0.37345 1.3705e-05 0.37404 -5.9163e-05 0.37115

0.36489 -0.00010715 0.34867 -6.8226e-06 -0.0010546

0.31336 -3.2669e-05 -0.0025193 -6.2761e-05 8.0462e-05

2.365e-05 0.00064313 0.00015504 -0.00020699 -0.00060666

-0.033722 -0.00012611 -2.7946e-05 0.00067263 -0.00048883

-0.31373 0.00014329 0.0012948 -0.56284 1.4923e-05

-0.6

6.5 插值拟合出$\Delta r(z), y(x)$

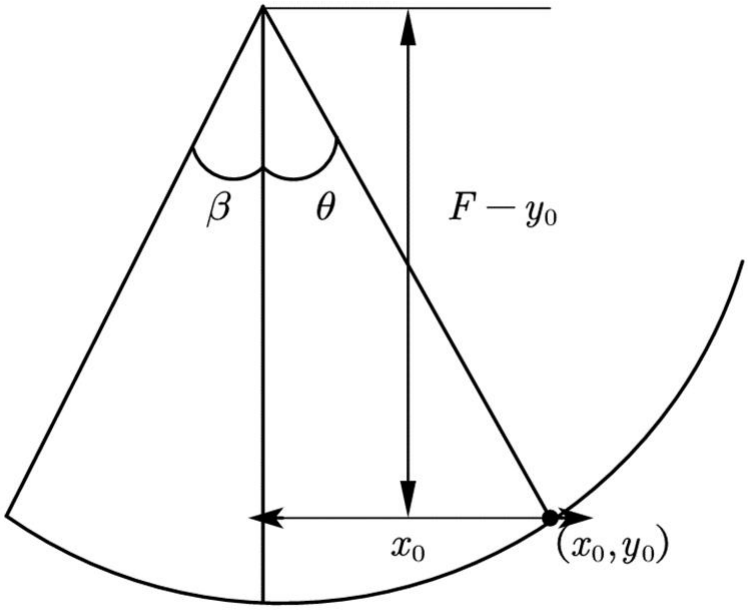
当我们优化结束以后，我们取得的是 100 个离散节点的最优调控方式——即 20 个离散点分别按照其对应优化出来的Δr结果伸缩调控。

我们先依据径向变化量将其最终坐标求解出来：

因为离散出来的 100 个点是按照等分 30 度角取点的，所以我们可得到每个点的对应角度;

$$\theta_i = i\frac{30^\circ}{100},\ (i = 1,2,3\sim100)$$

而由下图我们可以根据如图几何关系由每个点的径向伸缩量得到我们的新坐标



$$X_{inew}=X_i+\Delta r sin\theta_i \quad Y_{inew}=Y_i+\Delta r cos\theta_i$$

由此几何关系以及公式我们就得到了 100 个点本来的 x,y 值，伸缩后的 x,y 值，以及伸缩的长度Δr

那么我们依据各个点的Δr与初始 y 值（也是三维系中的 z 值）插值拟合出$\Delta r(z)$

由各个点的 y_new 与 x_new 插值拟合得出工作抛物线方程 y（x）

插值拟合步骤如下：

6.5.1 曲线插值

根据现有的方法，我们想到用使用三次样条插值曲线去得出连续变化分段曲线，三次样条插值保证在每一点连续且可导，且曲率数值也不会有较大的起伏变化，客观上相当符合曲线的曲率变化规律。

本方法运用三次多项式样条插值，通过在每个数据区间上逼近真实函数，在已知的数据点之间得到一个平滑的连续函数。

在每个数据区[xi,xi+1]内都分别定义多项式$S_i(x)$，多项式形式如下:

$$S_i(x)=a_i+b_i(x-x_i)+c_i(x-x_i)^2+d_i(x-x_i)^3$$

其中，x 是自变量（在这个案例中是沿光纤的位置），a_i,b_i,c_i,d_i是多项式系数，这些系数需要通过解线性方程组来找到，以确保函数在整个范围内平滑且连续。

其三次样条插值条件包括:$\begin{cases} S_i(x_i)=S_{i+1}(x_i) \\ S_i^{'}(x_i)=S_{i+1}^{'}(x_i) \\ S_i^{''}(x_i)=S_{i+1}^{''}(x_i) \end{cases}$

①方程一使函数值连续。

②方程二三保证了曲线在各个离散点x_i处的光滑性。

③边界条件可以是二阶导为零，或者固定的导数或者函数值。

6.5.2 曲线多项式拟合

如果只插值而不拟合，会破坏曲率函数的整体性，不易构造统一的方程，加大数据处理难度。如果只拿离散数据点直接来拟合方程，虽然多项式函数也具备连续且可导的性质，但是可能会在某一区域产生突变而造成偏差破坏了模型的现实意义，不一定符合实验数据。故我们先利用三次插条法分段建立平滑的分段曲线，再取得曲线上的多个点，从而扩增 6 个点为多个点，再运用 matlab 内置函数 ployfit 进行多项式拟合。

本步使用多项式拟来贴近插值函数并得出统一多项式表达式。m 组测量数据的 n 阶多项式最小二乘拟合可以看成是求解 n+1 个参数的线性方程组，方程组个数为 m 个，一般情况下测量数据个数 m 大于 n+1，所以，这个方程组为超定方程组，可以写为:

$$\begin{bmatrix} x_1^n & x_1^{n-1} & \dots & x_1 & 1 \\ x_2^n & x_2^{n-1} & \dots & x_2 & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_m^n & x_m^{n-1} & \dots & x_m & 1 \end{bmatrix} \begin{bmatrix} \widehat{\alpha}_n \\ \widehat{\alpha}_{n-1} \\ \vdots \\ \widehat{\alpha}_1 \\ \widehat{\alpha}_0 \end{bmatrix} = \begin{bmatrix} y_{s_1} \\ y_{s_2} \\ \vdots \\ y_{s_m} \end{bmatrix}$$

式中x_1~~x_m为测量点；y_{s1}~~y_{sm}为对应测量点的测量结果；a_0~~a_n为计算得到的拟合参数。可以将该矩阵简写为:

$$A\widehat{\alpha} = Y_s$$

这个超定方程组可以由广义逆矩阵求解，写成:

$$\hat{\alpha} = Pinv(A)$$
$$Y_s = KY_s$$

式中 Pinv（A）或者 K 为矩阵 A 的广义逆矩阵[4]。

于是可以求得 α 数矩阵，该矩阵在本题环境下使多项式在相应范围内与插值曲线贴合，达到得出统一表达式的目的。

最终我们运用三阶多项式拟合，得到拟合结果为：

$$y(x) = 1.29916e-06x^3 + 0.001551445x^2 + 0.005647238x - 300.562$$

$$\Delta r(\theta) = -55.6571\theta^4 + 36.6038\theta^3 - 8.8907\theta^2 + 2.676\theta - 0.3857$$

之后根据各个真实主索点对应极坐标系中的偏角θi带入公式计算出来各个节点的伸缩量Δr，再反空间变换回原空间坐标得到各个节点的对应伸缩量。结果储存于附件四当中

七， 问题三的求解

问题三是在问题二的基础上进一步对模型建立结果进行分析与对照，主要参照指标为：馈源舱的接受比，即在馈源舱接受范围内（直径为 1m 的圆盘）接收的信号与最终有效工作 300m 范围内接受的总电磁波信号的比。

那么我们也参照这评价标准建立模型来评估其接受比。

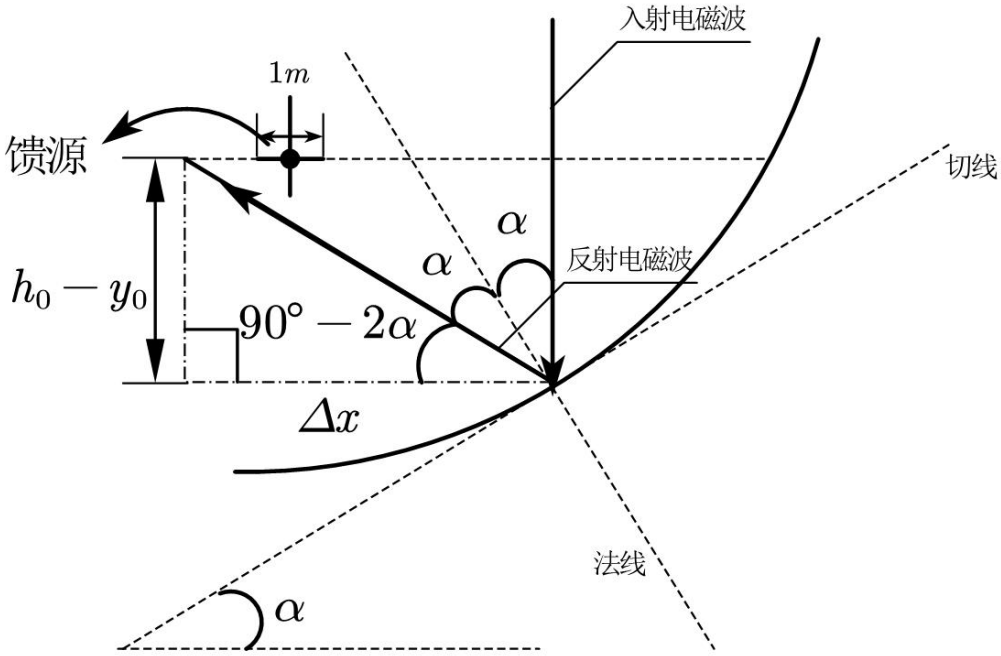
7.1 建立等效射线并进行反射几何分析

首先：我们由物理波动学可知，电磁波的传播规律符合反射定律，类同射线。故我们将这些电磁波等效抽象为等价的电磁射线来研究他们等同的反射性质。由此，我们便建立起等效射线模型，并且可以运用几何知识对其进行处理和分析。具体分析反射过程：

（1） 平行于馈源舱与基准球心连线（即理想抛物面对称轴）的平行电磁射线射入反射镜工作抛物曲面

（2） 在（问题二中降维平面化曲线）曲线上某点（x_0, y_0）处与反射面接触，以曲线在此点处切线为反射平面，垂直于切线且过该点的直线为法线，切向与“水平方向”夹角为α

（3） 因为入射电磁波是“竖直方向”，垂直于“水平方向”，又因为切向与法线同样是垂直关系，我们两组垂直关系可以得到入射电磁波与法线夹角为α，由反射定律可知，反射电磁线相对入射电磁线偏转 2α



7.2 建立数学模型计算接受比

由上述几何分析我们得到了反射的定量几何关系关联，而我们已经得到了工作抛物线的拟合方程，那么通过求导，每个点处的导数值我们也就确定了，也就是说每个点处的 tanα值是已知量。那么我们由每一个点的坐标以及切线与水平方向的夹角α，可以得到反射电磁线与水平方向夹角为$90^{\circ} - 2\alpha$具体数值，那么我们就自然而然可以得到**反射电磁线的具体运行轨迹**（有了起点与倾斜角）。

而由题目所说，若想要被馈源舱接收到信号，即反射电磁线可以达到馈源舱周围直径为 1m 的圆盘内，信号就可以被接收，那么我们转化一下：反射光线到达径向馈源舱同高度时与馈源舱中心距离在（-0.5m,0.5m）范围内，就可以被**馈源舱接受信号**。

那么由于对称性，我们可以只分析（0,150m）范围内，也就是以抛物线顶点建系只分析正半轴的接收比，而由对称性可知，全局接收比等于正半轴接受比。

由于电磁信号平行射入反射镜面时是按 x 轴均匀分布的，故我们只要求出使反射光线达馈源舱同径向高度时起始反射点（x，y）中$x \in A$（A 为区间集合）的 A 集合分布，后运用下公式即求解出接受比：

$$\frac{A\text{集合区间总长度}}{150m}$$

具体求解过程如下：

$\Delta h = h_{馈源舱} - y_0$反射节点径向距离馈源舱的高度差

$\tan\beta = \tan(90^\circ - 2\alpha) = \dfrac{1}{tan2\alpha}$反射光线与水平方向的夹角β

$\tan(90^\circ - 2\alpha) = \dfrac{\Delta h}{\Delta x} = \dfrac{1-\tan\alpha^2}{2\tan\alpha}$三角恒等变换

$\Delta x = \dfrac{\Delta h}{\tan(90^\circ - 2\alpha)} = \dfrac{2\Delta h\tan\alpha}{1-\tan\alpha^2}$反射光线到达馈源舱径向同高度点时的水平位移

$Y = \Delta x - x_0 = \dfrac{\Delta h}{\tan(90^\circ - 2\alpha)} - x_0$反射光线末节点距离馈源舱中心的距离

本问题我们在处理接收比时，由于计算机只能处理离散数据，故我们如下处理：我们先按照 0.0001 为步长离散（0,150）区间为 1500000 份点，每两个点之间水平间距为 0.1mm，这对于我们的问题的主要因素而言是极其微小的量值，故而我们可以认为离散点联络起来可以估计连续。
从而我们转化为：

$$接受比 = \frac{N_{符合终端Y值条件的反射点}}{N_{全部点}}$$

最终运用我们已得到的工作抛物线拟合方程以及基准圆的方程带入计算得到结果：
馈源舱工作抛物面接受比为：12.2094%
馈源舱基准反射球面接收比为：5.45084%

八， 模型的评价与优化

8.1 模型评价

8.1.1 模型优点:

该模型采用将三维曲面抽象为二维曲线并在二维空间中计算和评价相关参数从而达到了简化目的，该方法具有创新，大大降低了处理难度，同时也能满足数学方面严谨性。

8.1.2 模型缺点:

本文降维思路较依赖于理想球面与理想抛物面的高度对称性，但是实际上基准态球面与理想球面，工作抛物面与理想抛物面之间肯定存在差距，何况还是通过离散面板的调整来形成的非平滑三维面，此方法未能很好地顾及到这些现实问题。
并且存在以下误差
(1)通过遗传算法优化后的调整后的二维曲线样本点与理想抛物线之间依然存在 10^{-3} 的差距。

(2)直接用先插值后拟合来得出角度与调整距离的函数关系来计算各现实主索节点的伸缩量，该函数关系仅仅只满足了连续可导条件和数值接近条件，与理想真实函数关系存在误差。
(3)直接对函数曲线上的离散点进行最优化和计算接收效率时，直接使用二维拟合的理想平滑曲线来计算，但实际应用中是由面板组成的非平滑曲面。

8.2 模型优化

可直接在三维基准球面中直接考虑将主索节点的调整距离进行优化来拟合三维抛物面，此方法更加贴合现实应用。

九， 参考文献

本文章全内容自创，无参考文献

附录

注：本模型第一问结果包含于第二问计算中，故只呈现第二问代码，第二问用 MATLAB 编译，第三问用 python 编译。

第二问（第一问待求解的顶点变化参数为优化向量第一位优化参数）

```
clear;clc;
loc_table=readtable('附件 1.csv');
var_names={'node','x','y','z'};
loc_table.Properties.VariableNames = var_names;
aim_name={'x','y','z'};
loc_mat=table2array(loc_table(:,aim_name));
loc_mat=loc_mat(:,1:3);
R=mean(sqrt((loc_mat(:,1).^2+loc_mat(:,2).^2+loc_mat(:,3).^2)));
N_chorm=100;angles=linspace(0,pi/6,N_chorm);
[delta_rs,best_RMSE,final_equation]=select_best_alteration(angles,R);
angles2=linspace(min(angles),max(angles),100);
delta_rs2=spline(angles,delta_rs(2:end),angles2);
delta_r_p=polyfit(angles2,delta_rs2,4);
a=36.7595/180*pi;b=pi/2-78.169/180*pi;
tran_mat=[cos(b),0,sin(b);0,1,0;-sin(b),0,cos(b)]*...
    [cos(a),-sin(a),0;sin(a),cos(a),0;0,0,1];
for i=1:size(loc_mat,1)
    loc_mat2(i,:)=(tran_mat\loc_mat(i,:)')'; %#ok<SAGROW>
end
logical_arr=(sqrt(loc_mat2(:,1).^2+loc_mat2(:,2).^2)<=150);
nodes=loc_table(logical_arr,'node');
nodes=table2cell(nodes);
loc_working_mat=loc_mat(logical_arr,:);
theta=asin(sqrt(loc_working_mat(:,1).^2+loc_working_mat(:,2).^2)/R);
flex_length=polyval(delta_r_p,theta);
loc_working_mat2(:,1)=loc_working_mat(:,1)+flex_length.*sin(theta)...
    .*(loc_working_mat(:,1)./(sqrt(loc_working_mat(:,1).^2+loc_workin
g_mat(:,2).^2)));
```

```
loc_working_mat2(:,2)=loc_working_mat(:,2)+flex_length.*sin(theta)...
    .*(loc_working_mat(:,2)./(sqrt(loc_working_mat(:,2).^2+loc_workin
g_mat(:,2).^2)));
loc_working_mat2(:,3)=loc_working_mat(:,3)-flex_length.*cos(theta);
for i=1:size(loc_working_mat2,1)
    loc_working_mat2(i,:)=(tran_mat*loc_mat(i,:)')';
end
filename='附件4.xlsx';start_row=2;
loc_working_cell=num2cell(loc_working_mat2);
loc_working_cell_entire=nodes;
for i=1:3

loc_working_cell_entire=[loc_working_cell_entire,loc_working_cell(:,i
)]; %#ok<AGROW>
end
flex_length=num2cell(flex_length);
flex_cell=[nodes,flex_length];
peak_coordinate=(tran_mat*[0;0;-(300+delta_rs(1))])';
peak_coordinate=num2cell(peak_coordinate);
writecell(loc_working_cell_entire,filename,'Sheet','调整后主索节点编
号及坐标','Range',['A' num2str(start_row)]);%
writecell(flex_cell,filename,'Sheet','促动器顶端伸缩量','Range',['A'
num2str(start_row)]);%
writecell(peak_coordinate,filename,'Sheet','理想抛物面顶点坐标
','Range',['A' num2str(start_row)]);%









function
[delta_rs,best_RMSE,final_equation]=select_best_alteration(angles,R)
    Xs=R*sin(angles);Ys=-R*cos(angles);
    N = 1000;  %种群内个体数目
    N_chrom=size(angles,2)+1; %染色体节点数，也就是每个个体有多少条染
色体，其实说白了就是看适应函数里有几个自变量。
    iter = 1000; %迭代次数，也就是一共有多少代
    mut = 0.2;  %突变概率
    acr = 0.2; %交叉概率
```

```
    chrom_range=[-0.6,0.6];%每个节点的值的区间
    fitness_ave = zeros(1, iter);%存放每一代的平均适应度
    fitness_best = zeros(1, iter);%存放每一代的最优适应度
    % 初始化，这只是用于生成第一代个体，并计算其适应度函数
    chrom = Initialize(N, N_chrom, chrom_range); %初始化染色体
    fitness = CalFitness(chrom,N,Xs,Ys,R); %计算适应度
    chrom_best = FindBest(chrom, fitness, N_chrom); %寻找最优染色体
    fitness_best(1) = chrom_best(end); %将当前最优存入矩阵当中
    fitness_ave(1) = CalAveFitness(fitness); %将当前平均适应度存入矩阵
当中
    % 用于生成以下其余各代，一共迭代多少步就一共有多少代
    for t = 2:iter
        chrom = MutChrom(chrom, mut, N, N_chrom, chrom_range, t, iter); %
变异
        chrom = AcrChrom(chrom, acr, N, N_chrom); %交叉
        fitness = CalFitness(chrom,N,Xs,Ys,R); %计算适应度
        chrom_best_temp = FindBest(chrom, fitness, N_chrom); %寻找最优
染色体
        if chrom_best_temp(end)>chrom_best(end) %替换掉当前储存的最优
            chrom_best = chrom_best_temp;
        end
        %替换掉最劣
        [chrom, fitness] = ReplaceWorse(chrom, chrom_best, fitness);
        fitness_best(t) = -chrom_best(end); %将当前最优存入矩阵当中
        fitness_ave(t) = -CalAveFitness(fitness); %将当前平均适应度存
入矩阵当中
    end
    delta_rs=chrom_best(1:end-1);
    best_RMSE=chrom_best(end);
    figure(1)
    plot(2:iter, fitness_ave(2:end), 'r', 2:iter, fitness_best(2:iter),
'b')
    ylabel('RMSE')
    xlabel('iteration')
    title('graph of genetic optimization ')
    grid on
    legend('average RMSE', 'min RMSE')
    disp(['最优染色体为', num2str(chrom_best(1:end-1))])
    disp(['最优适应度为', num2str(chrom_best(end))])
    Xs2=Xs+delta_rs(2:end).*(Xs./R);
    Ys2=Ys-delta_rs(2:end).*(Ys./R);
    final_equation=polyfit(Xs2,Ys2,4);
    disp(['最终曲线方程',num2str(final_equation)]);
end
```

```
function chrom_new = Initialize(N, N_chrom, chrom_range)
    chrom_new = rand(N, N_chrom);
    for i = 1:N_chrom %每一列乘上范围
        chrom_new(:, i) = chrom_new(:,
i)*(chrom_range(2)-chrom_range(1))+chrom_range(1);
    end
end

function fitness = CalFitness(chrom,N,Xs,Ys,R)%计算适应度
    fitness = zeros(N, 1);
    for i = 1:N
        fitness(i) = CalRMSE(chrom(i,:),Xs,Ys,R);%%该函数是定义的适应
度函数，也可称为代价函数，用于以后筛选个体的评价指标
    end
end

function chrom_new = MutChrom(chrom, mut, N, N_chrom, chrom_range, t,
iter)%进行变异处理
    for i = 1:N %%N 是个体总数，也就是每一代有多少头袋鼠
        for j = 1:N_chrom  %N_chrom 是染色体节点数，就是有几条染色体
            mut_rand = rand; %随机生成一个数，代表自然里的基因突变，然
后用改值来决定是否产生突变。
            if mut_rand <=mut  %mut 代表突变概率，即产生突变的阈值，如
果小于 0.2 的基因突变概率阈值才进行基因突变处理，否者不进行突变处理
                mut_pm = rand; %增加还是减少
                mut_num = rand*(1-t/iter)^2;
                if mut_pm<=0.5
                    chrom(i, j)= chrom(i, j)*(1-mut_num);
                else
                    chrom(i, j)= chrom(i, j)*(1+mut_num);
                end
                chrom(i, j) = IfOut(chrom(i, j), chrom_range); %检验是
否越界
            end
        end
    end
    chrom_new = chrom;%%把变异处理完后的结果存在新矩阵里
end

function chrom_new = AcrChrom(chrom, acr, N, N_chrom)
    for i = 1:N
        acr_rand = rand;%生成一个代表该个体是否产生交叉的概率大小，用
于判别是否进行交叉处理
```

```
        if acr_rand>acr %如果该个体的交叉概率值大于产生交叉处理的阈值，
则对该个体的染色体（两条，因为此案例中有两个自变量）进行交叉处理
            acr_chrom = floor((N-1)*rand+1); %要交叉的染色体
            acr_node = floor((N_chrom-1)*rand+1); %要交叉的节点
            %交叉开始
            temp = chrom(i, acr_node);
            chrom(i, acr_node) = chrom(acr_chrom, acr_node);
            chrom(acr_chrom, acr_node) = temp;
        end
    end
    chrom_new = chrom;
end

function chrom_best = FindBest(chrom, fitness, N_chrom)
    chrom_best = zeros(1, N_chrom+1);
    [maxNum, maxCorr] = max(fitness);%因为所有个体对应的适应度大小都被
存放在 fitness 矩阵中
    chrom_best(1:N_chrom) =chrom(maxCorr, :);
    chrom_best(end) = maxNum;
end

function c_new = IfOut(c, range)
    if c<range(1) || c>range(2)
        if abs(c-range(1))<abs(c-range(2))
            c_new = range(1);
        else
            c_new = range(2);
        end
    else
        c_new = c;
    end
end


function [chrom_new, fitness_new] = ReplaceWorse(chrom, chrom_best,
fitness)
    max_num = max(fitness);
    min_num = min(fitness);
    limit = (max_num-min_num)*0.2+min_num;
    replace_corr = fitness<limit;
    replace_num = sum(replace_corr);
    chrom(replace_corr, :) = ones(replace_num, 1)*chrom_best(1:end-1);
    fitness(replace_corr) = ones(replace_num, 1)*chrom_best(end);
    chrom_new = chrom;
```

```
    fitness_new = fitness;
end

function fitness_ave = CalAveFitness(fitness)
    [N ,~] = size(fitness);
    fitness_ave = sum(fitness)/N;
end

function RMSE=CalRMSE(delta_r,Xs,Ys,R)
    a=(1-0.466)*R;p=2*(R-a+delta_r(1));
    Xs2=Xs+delta_r(2:end).*(Xs./R);
    Ys2=Ys-delta_r(2:end).*(Ys./R);
    Ys_ideal=(Xs2.^2-p^2-2*a*p)/(2*p);
    RMSE=-sqrt(sum((Ys_ideal-Ys2).^2)/size(Ys,2));
end
```

第三问

```
#引入最终工作抛物面拟合方程式
def f(x):
    return (1.29916*10**(-6))*x**(3)+0.001551445*x**2+0.005647238*x#
三次拟合工作抛物线方程
import numpy as np
import math
h0=0.466*300.4+0.3786#探测镜距离顶点距离·焦距的四分之一          参数
要改
x=np.arange(0,150,0.0001)#设置初始处理点，只考虑一半工作抛物面
y=f(x)#每个初始点的 y 值
n=len(x)
dy_dx=np.gradient(y, x)#每个点处的导数也就是 tanα
Li=[]
Lx=[]
N=0
for i in range(n):
    tan1=dy_dx[i]#tan(alpha)
    tan=(1-tan1**2)/(2*tan1)#tan(90-2alpha)
    delta_h=h0-y[i]
    delta_x=delta_h/tan
    Y=delta_x-x[i]
    idex=i
    nember=x[i]
    if -0.5<=Y<=0.5:
        Li.append(idex)#储存符合解的索引
        Lx.append(nember)
        N+=1#能射到接收点的数目
```

```
bili=N/n
print('工作抛物面有效区域接收到的反射信号与 300 米口径内反射面的反射信号之比为:',bili)
def circl(x):
    return 300-math.sqrt(90000-x**2)

x2=np.arange(0,250,0.0001)#设置初始处理点，只考虑一半基准圆面
n2=len(x2)
y2=[]
for i in range(n2):
    yi=circl(x2[i])
    y2.append(yi)

dy_dx2=np.gradient(y2, x2)#每个点处的导数也就是 tanα
Li2=[]
Lx2=[]
N2=0
for i in range(n2):
    tan1=dy_dx2[i]#tan(alpha)
    tan=(1-tan1**2)/(2*tan1)#tan(90-2alpha)
    delta_h=h0-y2[i]
    delta_x=delta_h/tan
    Y=delta_x-x2[i]
    idex=i
    nember=x2[i]
    if -0.5<=Y<=0.5:
        Li2.append(idex)#储存符合解的索引
        Lx2.append(nember)
        N2+=1#能射到接收点的数目
bili=N2/n2
print('基准球面有效区域接收到的反射信号与 500 米口径内反射面的反射信号之比为:',bili)
```